CloudGuard Online Deployment and GRC Update

CloudGuard Update

Online deployment + GRC

Status deck for shared E2E Dokploy deployment, GRC module, and demo/video flow.

What We Are Updating Today

Area	Update
Online deployment	Dokploy PoC package is ready and pushed to GitHub
Shared VM plan	CloudGuard can be migrated onto common E2E Dokploy VM
GRC	Internal CloudGuard GRC module is PoC-ready
Evidence flow	Scanner outputs can enter through `/api/evidence` connectors
Demo	Show compliance dashboard, operations view, evidence flow, vulnerable lab automation, and cost plan

---

Online Deployment Status

- Created a **Dokploy-ready Docker Compose stack** for CloudGuard
- Services included: backend/API/dashboard, PostgreSQL, MinIO, scheduler worker, scan worker, evidence connector
- Added `env.example`, deployment README, DNS plan, and validation steps
- Local checks passed for Dokploy compose configuration
- Latest changes pushed to GitHub branch: `codex/decoupled-scanner-deployment`

Dokploy CloudGuard stack

Figure 1: Dokploy CloudGuard stack

Pending: live E2E cloud-node deployment after VM credentials/access are available.

Dokploy PoC Shape

Shared E2E VM Onboarding Checklist

Check	Requirement
Docker compatibility	Project must run through Dockerfile or Docker Compose
Env variables	Provide `.env.example`; no secrets in GitHub
Resource estimate	CPU, RAM, disk, ports, database, storage, workers
Local proof	Screenshot/logs of local Docker run before Dokploy
Persistence	Define volumes/backups for DB, uploads, logs, reports
Health and monitoring	Health endpoint, logs, restart policy, uptime check

This checklist can be reused for Prudhvi, Ishaan, Anish, and future projects.

GRC Update

- Using a **custom CloudGuard GRC module** inside the portal for the PoC
- Compliance dashboard shows framework coverage, control status, evidence count, source summary, and recent evidence
- Evidence artifacts store `control_id`, source system, scanner type, checksum, raw payload, and metadata
- Findings can be mapped to CIS, NIST, ISO 27001, DPDP, OWASP, Kubernetes, and internal controls
- External scanners do not need to be inside the portal; they can post normalized evidence to one API

Live Demo Automation Requirement

Sir's requirement can be implemented as four safe demo buttons:

Button	What it should do
Deploy and scan vulnerable AWS	Create vulnerable resources in a sandbox AWS account, scan, collect evidence, destroy
Deploy and scan vulnerable GCP	Create vulnerable resources/project lab, scan, collect evidence, destroy
Deploy and scan vulnerable Kubernetes	Start a temporary kind/k3s vulnerable cluster or namespace, scan, delete
Deploy and scan vulnerable IaC	Load vulnerable Terraform/CloudFormation files, scan, store evidence

The UI label can still say "account", but technically we should create **ephemeral vulnerable labs/resources** with strict TTL and cleanup.

Recommended Demo Orchestrator Flow

```
User clicks "Show Live Demo"
        |
        v
Demo orchestrator creates vulnerable lab
        |
        v
Scanner runs automatically
        |
        v
Findings are posted as GRC evidence
        |
        v
Cleanup job destroys lab after scan or 5 minutes
```

```
creating_lab -> scanning -> ingesting_evidence -> destroying_lab -> completed
```

Safety Guardrails For Vulnerable Labs

- Use isolated sandbox accounts/projects only, never production credentials

- Prefer creating vulnerable resources over creating/closing full cloud accounts
- Enforce TTL cleanup at 5 minutes and a backup scheduled cleanup worker
- Add budget/quotas, region allowlist, and resource tags like `cloudguard-demo=true`
- Block public secrets and destructive permissions in demo templates
- Store demo evidence in GRC, then delete cloud resources immediately after scan

---

Why Not Create Full Accounts

Cloud	Reason
AWS	`CreateAccount` and `CloseAccount` are asynchronous, so account readiness/closure may not finish in a 5-minute demo
GCP	Project deletion has a recovery/deletion lifecycle, so it is not a clean instant close/open loop
Kubernetes	Full managed clusters are slower and costlier; local `kind`/`k3s` is faster for demos
IaC	No account is needed; scan vulnerable templates directly

Use **pre-created sandbox accounts/projects** and create only temporary vulnerable resources inside them.

What We Need To Build

Component	Responsibility
UI buttons	Trigger AWS, GCP, Kubernetes, or IaC demo lab
Demo lab APIs	Create `demo_labs` job and return live status
Demo worker	Runs Terraform/Pulumi/kubectl steps and starts scanner
Cleanup worker	Destroys expired resources every minute
Evidence ingestion	Posts findings to `/api/evidence` for GRC mapping
Audit trail	Stores status, scan job ID, cleanup result, and errors

---

Demo Lab Database Table

```
demo_labs(
  demo_id,
  provider,
  status,
  created_by,
  created_at,
  expires_at,
  scan_job_id,
  terraform_state_uri,
  cleanup_status,
  error
)
```

Status flow:

```
creating_lab -> scanning -> ingesting_evidence -> destroying_lab -> completed
```

Demo APIs And Workers

API / Worker	Purpose
`POST /api/demo-labs/aws/run`	Deploy AWS vulnerable lab, scan, destroy
`POST /api/demo-labs/gcp/run`	Deploy GCP vulnerable lab, scan, destroy
`POST /api/demo-labs/kubernetes/run`	Deploy temporary vulnerable cluster/namespace, scan, destroy
`POST /api/demo-labs/iac/run`	Scan vulnerable IaC files and store evidence
`demo_lab_worker.py`	Executes lab lifecycle
`demo_lab_cleanup_worker.py`	Enforces TTL cleanup and retries failed cleanup

Vulnerable Lab Templates

Lab	Example vulnerable components
AWS	Public-risk S3 config, open SSH security group, wildcard IAM policy, optional tiny EC2

Lab	Example vulnerable components
GCP	Public-risk storage bucket, public SSH firewall rule, overprivileged service account, optional tiny VM
Kubernetes	Privileged pod, `hostPath` mount, default token auto-mount, broad RBAC, NodePort, no NetworkPolicy
IaC	Vulnerable `.tf`, `.yaml`, `.json` templates scanned without deploying cloud resources

Templates must tag/label resources with `cloudguard-demo=true` and `expires_at`.

Cost Estimate Per Demo Run

Assumption: no EKS/GKE, no NAT Gateway, no load balancer, no large VM, cleanup within 5 minutes.

Demo	Expected cost per 5-minute run
AWS vulnerable resources	Usually below `$0.01`
GCP vulnerable resources	Usually below `$0.01`
Kubernetes on E2E VM	No extra cloud cost
IaC scan	No extra cloud cost
Full 4-button demo	Usually below `$0.05`

Safe monthly lab budget for PoC: **500- 1,000**, excluding the shared E2E VM cost.

Cost Controls

- Avoid EKS/GKE for live demo; use `kind/k3s`
- Avoid NAT Gateway and managed load balancers in demo templates
- Use tiny/spot/free-tier-sized VMs only when absolutely needed
- Auto-destroy resources after scan completion or 5-minute TTL
- Add cloud budgets, alerts, region allowlist, and service quotas
- Run a daily cleanup script for any stale `cloudguard-demo=true` resources

GRC evidence flow

Figure 2: GRC evidence flow

Evidence Pipeline Into GRC

Demo / Video Flow

```
Open CloudGuard dashboard
        |
        v
Show Compliance / GRC page
        |
        v
Show Operations page: jobs + evidence artifacts
        |
        v
Show connector contract: POST /api/evidence
        |
        v
Show one evidence item mapped to a control
```

Target video length: **90-120 seconds**.

What Is Ready vs Pending

Ready	Pending
Dokploy compose stack	Live VM deployment
GRC dashboard and evidence views	Real deployed URL
Evidence connector service	Domain/subdomain mapping
Marp/PPT update decks	Production monitoring setup
GitHub branch for collaboration	Demo-lab APIs and cleanup worker
Vulnerable lab design	Real scanner-output demos

Immediate Ask

Provide shared E2E VM access, domain/subdomain decision, and sample scanner outputs.

Then we can deploy CloudGuard on Dokploy, record the GRC demo video, and onboard the next project using the same checklist.